



PIZZA PALACE

Software Requirements Specification

MERN STACK · ONLINE FOOD ORDERING PLATFORM · STUDENT EDITION

React

Node.js

Express

MongoDB

JWT

Tailwind

DOCUMENT TYPE

SRS — v1.0

Student Developer Edition

TECH STACK

MERN + JWT

Full-Stack Web App

YEAR

2026

Academic Project

Table of Contents

#	Section	Page
1	Introduction	3
2	Project Overview	3
3	Business Objectives	3
4	Scope of the System	4
5	User Roles and Responsibilities	4
6	Functional Requirements	5
7	Non-Functional Requirements	6
8	System Architecture	7
9	Frontend Design	8
10	Backend Design	9
11	Database Design	10
12	Authentication & Authorization	11
13	API Specifications	12
14	UI/UX Requirements	13
15	Security Requirements	13
16	Testing Strategy	14
17	Deployment Strategy	15
18	Future Enhancements	15
19	Conclusion	16

1. Introduction

Pizza Palace is a modern, full-stack online food ordering platform built on the MERN stack (MongoDB, Express.js, React, Node.js). It enables customers to seamlessly browse pizza catalogues, manage their cart, place orders, and track delivery status — all through an intuitive, mobile-responsive interface secured by JWT-based authentication.

This Software Requirements Specification (SRS) document formally defines the functional and non-functional requirements, system architecture, database design, security strategy, API contracts, and deployment plan for the Pizza Palace platform. It serves as the master reference document for developers, QA engineers, and stakeholders throughout the project lifecycle.

Document Purpose

This SRS is intended to provide a complete and unambiguous description of Pizza Palace so that all involved parties have a shared understanding of what the system must do, how it must behave, and the constraints it must operate within.

Attribute	Details
Document Title	Pizza Palace — Software Requirements Specification
Version	1.0
Status	Approved for Development
Tech Stack	React · Node.js · Express.js · MongoDB · JWT · Tailwind CSS
Deployment Targets	Vercel (Frontend) · Render (Backend) · MongoDB Atlas (DB)
Classification	Student Developer Edition

2. Project Overview

Pizza Palace is a production-grade, end-to-end web application that replicates the real-world operation of an online pizza restaurant. It is structured as a single-page application (SPA) on the client side and a RESTful API server on the backend, communicating over HTTPS using JSON payloads.

Component	Technology	Responsibility
Frontend	React + Tailwind CSS	User Interface, State Management, Routing
Backend	Node.js + Express.js	REST API, Business Logic, JWT Middleware
Database	MongoDB + Mongoose	Data Persistence, Schema Validation, Relations
Auth	JWT + bcryptjs	Secure Login, Role-Based Access Control
Hosting	Vercel / Render / Atlas	Production Deployment, CI/CD, Cloud DB

3. Business Objectives

Pizza Palace aims to modernise and automate the complete pizza ordering value chain. The following strategic objectives drive the platform design and feature prioritisation:

■ Digitise Order Workflows	Eliminate manual telephone-based ordering, reducing processing errors and order lead time by automating the customer → kitchen pipeline.
■ Centralised Order Management	Provide administrators with a single dashboard to manage incoming orders, update statuses, and control the pizza catalogue in real time.
■ Superior Customer Experience	Deliver a fast, visually engaging, mobile-responsive storefront enabling customers to browse, customise, and track orders with zero friction.
■ Security & Compliance	Implement industry-standard security (JWT, bcrypt, HTTPS) to protect customer data and ensure only authorised access to admin functions.
■ Scalability & Growth	Design the system for horizontal scalability so that increased traffic and product catalogue growth can be accommodated without architectural changes.
■ Foundation for Innovation	Establish a clean API-first architecture that enables future integrations such as payment gateways, AI recommendations, and delivery tracking.

4. Scope of the System

The Pizza Palace platform covers the complete lifecycle of an online pizza order from customer registration through to order delivery confirmation. The scope is divided into in-scope and explicitly out-of-scope items to maintain project focus.

4.1 In-Scope Modules

- Customer self-registration and secure login via JWT
- Pizza browsing with category filters, search, and detail view
- Shopping cart — add, update quantity, remove items
- Order placement with delivery address capture
- Order history and real-time status tracking for customers
- Admin panel: pizza catalogue CRUD (create, read, update, delete)
- Admin panel: order management and status update workflow
- Role-based route protection (customer vs administrator)
- Responsive UI for mobile, tablet, and desktop browsers
- Secure API with input validation and error handling

4.2 Out-of-Scope (Phase 1)

- Payment gateway integration (planned for Phase 2)
- Real-time GPS delivery tracking
- Third-party OAuth (Google / Facebook login)
- Multi-restaurant / multi-branch support
- Mobile native apps (iOS / Android)
- AI-based product recommendation engine

5. User Roles and Responsibilities

Role	Access Level	Capabilities
■ Customer	Standard	Register · Login · Browse Pizzas · Manage Cart · Place Orders · View Order History · Update Profile
■■ Administrator	Elevated	All Customer capabilities + Add/Edit/Delete Pizzas · View All Orders · Update Order Status · Manage Users
■ System	Internal	Token validation · Auto order-status transitions · Data validation middleware · Error logging

6. Functional Requirements

6.1 Authentication Module

ID	Requirement Description
FR-AUTH-01	The system shall allow new users to register with name, email, and password.
FR-AUTH-02	Passwords shall be hashed using bcryptjs (salt rounds ≥ 10) before storage.
FR-AUTH-03	On successful login, the system shall issue a signed JWT with role and expiry claims.
FR-AUTH-04	Protected API routes shall reject requests with missing or expired tokens with HTTP 401.
FR-AUTH-05	Admin-only routes shall reject non-admin tokens with HTTP 403.

6.2 Pizza Catalogue Module

ID	Requirement Description
FR-PIZ-01	The system shall display all available pizzas with name, image, description, category, and price.
FR-PIZ-02	Customers shall be able to filter pizzas by category (e.g., Veg, Non-Veg, Specialty).
FR-PIZ-03	Admins shall be able to create, update, and delete pizza records via the admin panel.
FR-PIZ-04	Each pizza record shall include an availability toggle to show/hide from the menu.
FR-PIZ-05	Pizza images shall be stored as URL references in MongoDB.

6.3 Cart Module

ID	Requirement Description
FR-CART-01	Authenticated customers shall be able to add pizzas to their cart with quantity selection.
FR-CART-02	The cart shall persist across page reloads using Context API + localStorage.
FR-CART-03	Customers shall be able to update quantities or remove items from the cart.
FR-CART-04	The cart shall display the real-time order total including item subtotals.

6.4 Order Management Module

ID	Requirement Description
FR-ORD-01	Customers shall be able to place orders from their cart by confirming a delivery address.
FR-ORD-02	Each order shall be stored with status: Pending → Confirmed → Preparing → Out for Delivery → Delivered.
FR-ORD-03	Customers shall be able to view their full order history with timestamps and statuses.

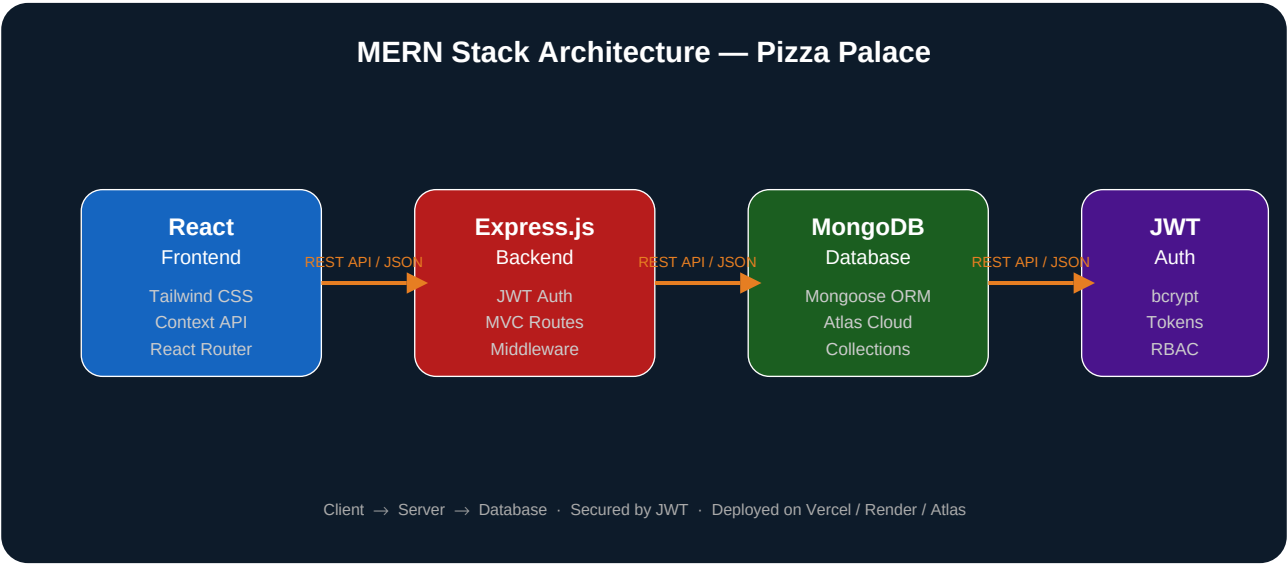
FR-ORD-04	Admins shall be able to view all orders and update each order's status.
FR-ORD-05	Order cancellation shall be permitted only when status is Pending.

7. Non-Functional Requirements

Category	ID	Requirement	Target Metric
Performance	NFR-01	API response time for standard reads	< 300 ms p95
Performance	NFR-02	Frontend initial page load (LCP)	< 2.5 s
Scalability	NFR-03	Concurrent users supported without degradation	≥ 500 users
Availability	NFR-04	Platform uptime SLA	99.5% monthly
Security	NFR-05	JWT token expiry window	24 hours
Usability	NFR-06	Mobile responsiveness — viewport support	320 px – 2560 px
Maintainability	NFR-07	Code coverage — unit + integration tests	≥ 70%
Compliance	NFR-08	Sensitive config via environment variables only	100% compliant

8. System Architecture

Pizza Palace follows a classic three-tier client-server architecture. The presentation tier (React SPA) communicates exclusively with the application tier (Express.js REST API) over HTTPS/JSON. The data tier (MongoDB Atlas) is never directly accessible from the browser — all data access is mediated through the backend API and validated by Mongoose schemas.



8.1 Backend Directory Structure

Folder / File	Purpose
<code>server.js</code>	Entry point — initialises Express, connects MongoDB, registers routes

<code>routes/</code>	Route definitions for auth, pizzas, orders (authRoutes, pizzaRoutes, orderRoutes)
<code>controllers/</code>	Business logic handlers for each resource — thin, testable functions
<code>models/</code>	Mongoose schemas: User, Pizza, Order — with validation and indexes
<code>middleware/</code>	JWT verifier, role guard, request logger, centralised error handler
<code>.env</code>	Environment variables: MONGO_URI, JWT_SECRET, PORT, NODE_ENV

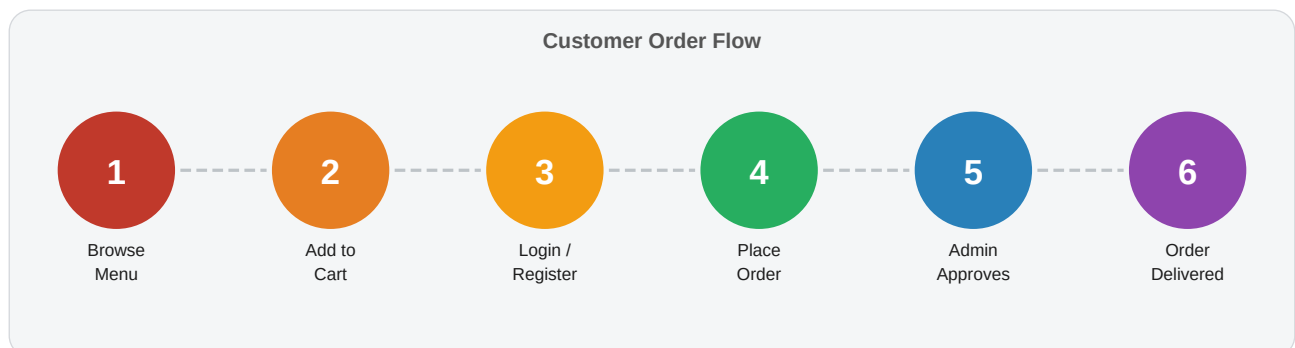
9. Frontend Design

The React frontend is structured as a component-driven SPA. State is managed globally using React Context API (AuthContext for user session, CartContext for cart state). Navigation is handled by React Router DOM v6 with protected routes for authenticated sections.

9.1 Key Pages & Components

Page / Component	Route	Access	Description
Home	/	Public	Hero banner, featured pizzas, CTA to menu
Menu	/menu	Public	Full pizza catalogue with category filter and search bar
Pizza Detail	/pizza/:id	Public	Full description, size options, add-to-cart control
Cart	/cart	Auth	Item list, quantity controls, order total, checkout trigger
Checkout	/checkout	Auth	Delivery address form, order summary, place order button
Order History	/orders	Auth	List of past orders with status badges and timestamps
Login / Register	/auth	Public	Tabbed auth form — JWT stored in localStorage on success
Admin Dashboard	/admin	Admin	Order overview, revenue summary, quick actions panel
Admin Pizzas	/admin/pizzas	Admin	Pizza CRUD table with inline edit and availability toggle
Admin Orders	/admin/orders	Admin	All orders table with status dropdown update controls

9.2 Order Flow Visualisation



10. Backend Design

The Express.js backend follows the MVC (Model-View-Controller) pattern. Routes delegate to controllers which contain business logic, while Mongoose models encapsulate data access. All routes are prefixed with /api and respond with structured JSON.

10.1 Middleware Chain

Middleware	Purpose	Applied To
express.json()	Parses incoming JSON request bodies	All routes
cors()	Enables cross-origin requests from allowed frontend origins	All routes
verifyToken	Validates JWT from Authorization header; attaches req.user	Protected routes
isAdmin	Checks req.user.role === 'admin'; returns 403 if not	Admin routes
errorHandler	Centralised error formatting — returns consistent error JSON	Global (last)
morgan()	HTTP request logger for development and debugging	Dev environment

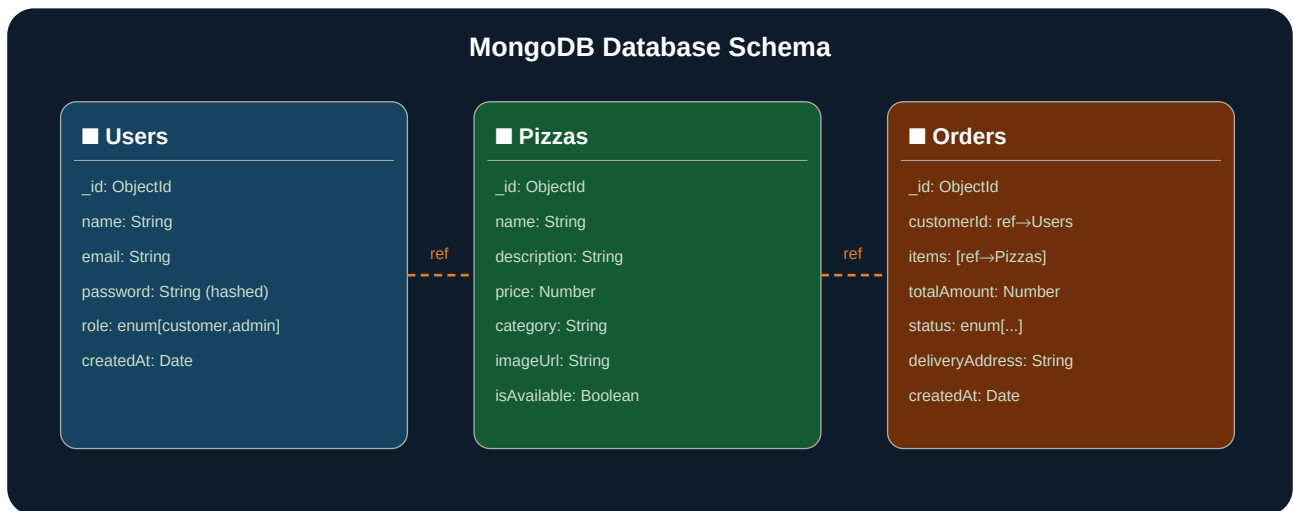
10.2 Error Response Convention

Standard Error Format

All API errors follow the structure: { success: false, message: '...', statusCode: 4xx/5xx }. Validation errors include a 'errors' array with field-level detail. HTTP status codes strictly follow REST conventions (200, 201, 400, 401, 403, 404, 500).

11. Database Design

Pizza Palace uses MongoDB as its primary database, accessed through Mongoose ODM. Three core collections form the data model: Users, Pizzas, and Orders. The Orders collection uses MongoDB ObjectId references to link to Users and Pizzas.



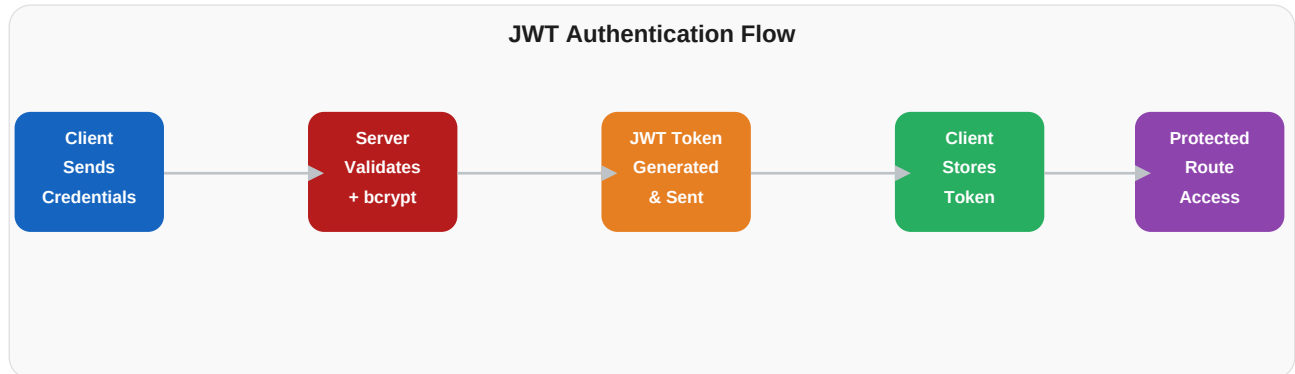
11.1 Collection Schemas

Field	Type	Constraints	Notes
— Users Collection —			
name	String	required, trim	Customer/admin display name
email	String	required, unique, lowercase	Used as login identifier
password	String	required, minLength: 6	Stored as bcrypt hash — never plaintext
role	String	enum: ['customer','admin']	Default: 'customer'
createdAt	Date	default: Date.now	Auto-set on document creation
— Pizzas Collection —			
name	String	required, trim	Pizza display name on menu
description	String	required	Short marketing description
price	Number	required, min: 0	Price in INR
category	String	required	e.g., Veg, Non-Veg, Specialty
imageUrl	String	required	CDN or S3 URL for pizza image
isAvailable	Boolean	default: true	Controls menu visibility
— Orders Collection —			
customerId	ObjectId	ref: 'User', required	Links order to registered customer
items	[[pizza, qty]]	required, min 1 item	Array of pizza references with quantities
totalAmount	Number	required	Computed from cart at order placement

status	String	enum: see flow	Pending → Confirmed → Preparing → Delivered
deliveryAddress	String	required	Full delivery address as free text

12. Authentication & Authorization

Security is implemented at multiple layers using industry-standard mechanisms. Passwords are never stored in plaintext. All sensitive routes require a valid JWT, and admin-only routes enforce role-based access control.



Mechanism	Implementation	Configuration
Password Hashing	bcryptjs library hashes password on registration	saltRounds = 12
Token Generation	jsonwebtoken.sign() called on successful login	HS256 algorithm
Token Payload	{ id, email, role } — no sensitive data	Signed with JWT_SECRET
Token Expiry	Tokens expire automatically; client must re-login	expiresIn: '24h'
Token Transport	Sent via Authorization: Bearer header	HTTP Header
Role Guard	isAdmin middleware checks decoded token role field	role === 'admin'
Token Storage	Client stores token in localStorage; cleared on logout	Frontend

13. API Specifications

All API endpoints follow RESTful conventions. The base URL is /api. Requests and responses use application/json content type. Protected endpoints require an Authorization header with a valid Bearer token.

Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register a new customer account
POST	/api/auth/login	Public	Login and receive JWT token
GET	/api/auth/profile	Auth	Fetch current logged-in user profile
PUT	/api/auth/profile	Auth	Update current user's profile details

Pizza Catalogue Endpoints

Method	Endpoint	Auth	Description
--------	----------	------	-------------

GET	/api/pizzas	Public	Fetch all available pizzas (supports ?category filter)
GET	/api/pizzas/:id	Public	Fetch single pizza by ID
POST	/api/pizzas	Admin	Create a new pizza record
PUT	/api/pizzas/:id	Admin	Update pizza details or toggle availability
DELETE	/api/pizzas/:id	Admin	Permanently delete a pizza from catalogue

Order Endpoints

Method	Endpoint	Auth	Description
POST	/api/orders	Auth	Place a new order with cart items and delivery address
GET	/api/orders/my	Auth	Retrieve current customer's order history
GET	/api/orders	Admin	Retrieve all orders across all customers
PUT	/api/orders/:id/status	Admin	Update the status of a specific order
DELETE	/api/orders/:id	Auth	Cancel a pending order (customer or admin)

14. UI/UX Requirements

The Pizza Palace UI is designed to be intuitive, visually engaging, and fully responsive across all device sizes. Tailwind CSS utility classes drive a consistent design system with a warm red/orange brand palette.

Requirement	Specification
Colour Palette	Primary: #C0392B (Red) · Secondary: #E67E22 (Orange) · Neutral: #F8F9FA, #1C1C1C
Typography	Headings: Inter Bold · Body: Inter Regular · Code: JetBrains Mono
Breakpoints	sm: 640px · md: 768px · lg: 1024px · xl: 1280px (Tailwind defaults)
Loading States	Skeleton loaders on all data-fetch operations — no blank screens
Toast Notifications	Success / error toasts for all user actions (add to cart, place order, login)
Empty States	Illustrated empty states with action CTA for cart, orders, and search results
Accessibility	WCAG 2.1 AA compliance — keyboard nav, ARIA labels, focus rings, alt text
Animation	Subtle Framer Motion transitions — page enters, cart updates, modal opens

15. Security Requirements

■ Password Security

All passwords hashed with bcryptjs before being stored in MongoDB. Raw passwords are never logged or stored. Minimum password length is 6 characters.

■ JWT Authentication

Tokens are signed with HS256 using a secret stored in your .env file. Expiry is set to 24h. The token is validated on every protected request by the verifyToken middleware.

■ Environment Variables

Your database URI, JWT secret, and any credentials should always live in a .env file. Never hardcode secrets or commit .env to Git — add it to .gitignore.

■ Input Validation

Validate all request bodies server-side using express-validator (or manual checks) before hitting controller logic. Reject invalid inputs early with a 400 status and clear error message.

■ CORS Configuration

Configure CORS to only allow requests from your frontend's origin. Use the cors() middleware with an explicit origin whitelist so other domains can't call your API.

■ HTTP Security Headers

Use the helmet.js middleware to automatically set security-relevant HTTP headers such as X-Content-Type-Options, X-Frame-Options, and Content-Security-Policy.

16. Testing Strategy

Pizza Palace adopts a layered testing approach covering unit, integration, and end-to-end testing to ensure correctness at every layer of the stack.

Test Type	Scope	Tools	Coverage Target
Unit Tests	Controller functions, utility helpers, Mongoose validators	Jest + Supertest	≥ 70%
Integration Tests	Full API route tests against a test MongoDB instance	Jest + MongoDB Memory Server	All endpoints
Frontend Tests	Component rendering, user interaction, Context state	React Testing Library + Vitest	Key user flows
E2E Tests	Complete user journeys: register → order → status update	Playwright / Cypress	Critical paths
Security Tests	JWT tampering, SQL/NoSQL injection, CORS bypass attempts	Manual + OWASP ZAP	All auth flows
Performance Tests	API response time under 500 concurrent users	k6 / Artillery	< 300 ms p95
Responsive Tests	UI layout verification across 5 device breakpoints	BrowserStack	All breakpoints

17. Deployment Strategy

Layer	Platform	Configuration	Notes
Frontend	Vercel	Auto-deploy from GitHub main branch	CDN edge distribution globally
Backend API	Render	Web Service — Node.js runtime, auto-scale	Zero-downtime deploys
Database	MongoDB Atlas	M10+ cluster, backups enabled, VPC peering	Multi-region failover
Environment Vars	Render Secrets	MONGO_URI, JWT_SECRET, NODE_ENV=production	Never committed to Git
CI/CD Pipeline	GitHub Actions	Lint → Test → Build → Deploy on push to main	Blocks deploy on test failure

18. Future Enhancements

Phase	Enhancement	Description	Priority
Phase 2	Payment Gateway	Razorpay / Stripe integration for online card and UPI payments	■ High

Phase 2	Google OAuth	One-click Google sign-in using Passport.js OAuth 2.0 strategy	■ High
Phase 2	SMS/Email OTP	Twilio/SendGrid OTP verification on registration and checkout	■ Medium
Phase 3	Live Order Tracking	Real-time GPS delivery tracking with Socket.IO and Google Maps API	■ Medium
Phase 3	AI Recommendations	ML-based pizza recommendation engine using order history and preferences	■ Low
Phase 3	Loyalty Program	Points-based rewards system with redemption on future orders	■ Low
Phase 4	Mobile Apps	React Native iOS and Android apps with push notifications	■ Low

19. Conclusion

Pizza Palace is a comprehensive, production-ready MERN stack application that demonstrates real-world full-stack engineering practices. It covers every aspect of modern web application development — from secure authentication and role-based access control, through a clean RESTful API architecture, to a polished React frontend with responsive design.

The system's modular architecture ensures that future enhancements — including payment gateways, AI-driven recommendations, and native mobile applications — can be integrated without architectural overhaul. The layered security model, comprehensive testing strategy, and cloud-native deployment pipeline position Pizza Palace as a scalable foundation for a real commercial food-ordering business.

This SRS document serves as your master reference while building Pizza Palace. Revisit it as you complete each module to keep your implementation aligned with the requirements.

■ Developer Quick Reference

Stack: React + Tailwind CSS · Node.js + Express.js · MongoDB + Mongoose · JWT

Run backend: `npm run dev` · Run frontend: `npm start` · DB: MongoDB Atlas (set `MONGO_URI` in `.env`)

Base API URL: `http://localhost:5000/api` · Auth header: `Authorization: Bearer <token>`